

Patching the Past: Efficient and Reversible Federated Unlearning via Low-Rank Adaptation

Ankit Sharma
AI25RESCH11004

Yedu Krishna P
AI25RESCH11003

Abstract

Federated Learning (FL) [13, 23] is an recent paradigm for privacy-preserving decentralized training, but it faces significant challenges complying with the “Right to be Forgotten.” Traditional Federated Unlearning methods [22] are computationally prohibitive, often requiring full-model retraining or suffering from catastrophic forgetting and privacy leakage vulnerabilities. Furthermore, transmitting full-model updates during the unlearning process incurs massive communication overhead. In this paper, we propose a novel and efficient federated unlearning framework leveraging Low-Rank Adaptation (LoRA) [15]. Rather than modifying the core global weights, our approach freezes the backbone and learns a modular “negative patch” through adapter layers to selectively neutralize the influence of the data targeted for deletion. We introduce a progressive three-stage unlearning strategy: (i) Targeted gradient ascent to destroy specific class memories, (ii) Lightweight performance recovery on unaffected classes, and (iii) Dual-loss optimization phase to simultaneously preserve retained knowledge and suppress the forgotten data. Evaluated on the CIFAR-10 [18] dataset, our method demonstrates rapid and stable unlearning that successfully defends against Membership Inference Attacks (MIA) while maintaining high utility on the remaining data. Furthermore, by restricting updates to low-rank matrices, our framework drastically reduces server-side aggregation costs and uniquely enables reversible unlearning by simply discarding the targeted adapters.

1. Introduction

In the era of big data, Federated Learning (FL) [23] has emerged as a transformative paradigm for training robust machine learning models while preserving data privacy. By allowing decentralized clients to collaborate and contribute to learning a shared global model without exchanging raw data, FL addresses the problems of data islands and privacy.

This way, knowledge is shared across models without having to share data directly, allowing clients to preserve their privacy.

Despite its privacy-preserving nature, FL faces a critical challenge: once a client contributes to the global model, their data is inherently embedded in the model’s weights. Recent legal frameworks now mandate the “Right to be Forgotten” enacted by regulatory laws such as the General Data Protection Regulation (GDPR) which grants users the authority to request the permanent erasure of their data from any trained system. In a centralized setting, where there is a single model, this is achieved by retraining the model from scratch or performing sophisticated Machine Unlearning [19] processes, but this is computationally prohibitive and practically unfeasible in a massive, decentralized FL network.

The process of removing a client’s influence from a global model, known as Federated Unlearning [16, 22, 28], is inherently difficult. There are two approaches to federated unlearning: (i) Exact unlearning [2] produces a model that is statistically indistinguishable from one retrained from scratch on the dataset with the forget data removed and (ii) Approximate unlearning [16, 22] methods seek to produce a model whose behavior is sufficiently close to an exactly unlearned model without the cost of full retraining. Exact Unlearning ensures perfect privacy but sacrifices efficiency. Conversely, Approximate Unlearning methods attempt to “subtract” a client’s contribution using gradient-based techniques. However, these often suffer from “Catastrophic Forgetting”, where the model’s general utility on remaining clients collapses, or Privacy Leakage, where the model still responds differently to the forgotten data, leaving it vulnerable to Membership Inference Attacks (MIA).

The above methods also have limited efficiency as they require the clients to fine-tune the entire model and then send them to the server for aggregation and distribution, all of which involves operations on millions of parameters if the architecture is large. This raises the question: **Can we improve the efficiency of federated unlearning algorithms? And more importantly, can we do this without**

compromising on the quality of unlearning?

In this paper, we propose an efficient unlearning framework based on **Low-Rank Adaptation (LoRA)** [14]. Rather than attempting to modify the core global weights, we introduce a method to learn a modular negative patch that targets the specific influence of the samples that are to be forgotten. By optimizing a dual-objective loss where one focusing on maximizing the entropy of the forget set and another on preserving the knowledge of the retain set, creating an interference signal that effectively blinds the model to the target data during inference. This approach allows for rapid, stable unlearning without the need for full-model fine-tuning or expensive retraining.

The primary contribution of this work is introducing LoRA to erase the client’s influence by updating only a fraction of the model parameters. This approach allows the server to perform the seamless aggregation of unlearning updates across multiple clients as the backbone network is frozen. We evaluated our method on the CIFAR-10 dataset and find that the proposed method preserves model utility by maintaining good test set accuracy while the adapters effectively negate the model’s performance on the samples that are to be forgotten. The use of adapters also allow for a reversible unlearning strategy by dropping the LoRA adapters if and when the clients decide to include the forget set back into the knowledge pool.

2. Related Work

Federated Learning. The foundational work in federated learning (FL), *FedAvg* [32], introduced a framework that aggregates locally trained model updates to construct a global model. Since then, FL has been widely adopted in privacy-sensitive domains such as healthcare [21, 24], finance [12, 26], and recommendation systems [29]. Despite its success, FL faces several challenges [8, 10, 17], among which data heterogeneity is the most fundamental issue [9, 20]. In practice, client data distributions are often non-independent and identically distributed (Non-IID), which significantly impacts model performance. This issue also extends to federated unlearning settings.

Machine Unlearning. Machine unlearning was first formalized by Cao et al. [4] as a means to efficiently and accurately remove specific information from trained models, thereby safeguarding data privacy [11]. A straightforward approach is to delete the target data and retrain the model from scratch. To improve efficiency, *SISA* [2] proposes a trade-off between storage and unlearning time. Other approaches modify trained models using precomputed parameters to adjust their behavior [1], or selectively remove the influence of certain data by pruning gradients [3]. However, most existing unlearning methods are designed for centralized settings and struggle to address the unique challenges

Conceptual Framework for Federated Machine Unlearning

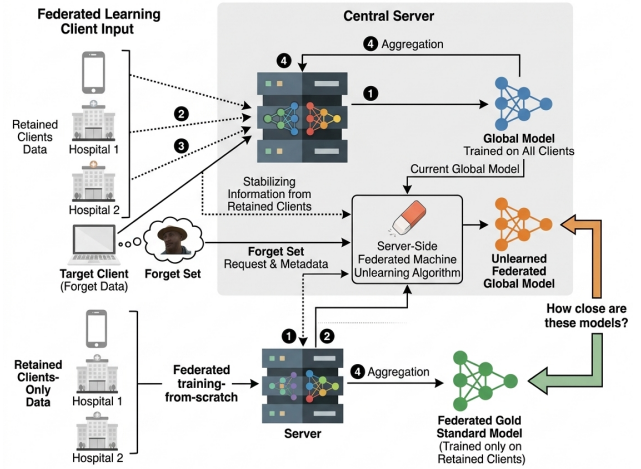


Figure 1. Federated Unlearning Framework.

of FL, such as restricted data access and Non-IID distributions.

Federated Unlearning. Federated unlearning (FU) methods (fig. 1) can be broadly categorized into three levels based on their targets [13, 22]: client-level, sample-level, and class-level. Client-level FU aims to remove the contribution of specific clients from the global model [6], while sample-level FU focuses on eliminating the influence of particular data subsets [5]. Class-level FU, on the other hand, seeks to erase knowledge of specific classes from the model’s decision boundary [27].

Recent approaches primarily identify and modify model parameters associated with the data to be forgotten. For instance, FUDP [27] prunes channels most relevant to target classes, while FFMU [5] leverages nonlinear functional theory to enhance efficiency without sacrificing unlearning quality. FedRecovery [30] employs differential privacy to remove client contributions by eliminating gradient residuals. FUMD [31] uses a randomly initialized degradation model to erase class-specific information, whereas FedAU [7] introduces an auxiliary classifier combined with linear transformations for unlearning. FedOSD [25] further improves efficiency by optimizing along orthogonal steepest descent directions, reducing performance degradation.

3. Proposed Methodology

Our proposed framework for Federated Unlearning using Low-Rank Adaptation (LoRA) treats unlearning as a parameter-efficient optimization problem where a negative patch is learned to neutralize the influence of specific data without altering the core global weights. We use models from a setup that was trained in accordance to traditional federated learning approaches (specifically FedAVG [32]).

We now describe the typical federated learning algorithm:

Federated Learning

Let the federated learning network consist of K clients. Each client $k \in \{1, \dots, K\}$ holds a private local dataset $\mathcal{D}_k$ of size $n_k = |\mathcal{D}_k|$. The total number of samples across all participating clients is $N = \sum_{k=1}^K n_k$. Each client trains on their private local set during one round of training for a set number of local epochs and at the end of the round, every client communicates their updates/parameters to the server for aggregation after which the same set of parameters are forwarded to each client for initializing the local model to train them in the following round. The training process over T communication rounds is mathematically formulated as follows:

1. Local Loss Function

For a model with parameters w , the local loss function $L_k(w)$ for client k is the average error over its dataset:

$$L_k(\theta) = \frac{1}{n_k} \sum_{(x,y) \in \mathcal{D}_k} \ell(\theta; x, y) \quad (1)$$

where $\ell(w; x, y)$ is the loss (e.g., cross-entropy) for a single data sample (x, y) .

2. Local Model Update

At the start of round t , the central server sends the current global model parameters θ^t to all clients. Each client initializes its local model as $\theta_k^t = w^t$. The client then updates its model by taking one or more gradient descent steps on its local loss L_k , with a learning rate η :

$$\theta_k^{t+1} = \theta_k^t - \eta \nabla L_k(\theta_k^t) \quad (2)$$

This produces the updated local parameters θ_k^{t+1} .

3. Global Model Aggregation

After local training, the clients send their updated parameters $\{\theta_k^{t+1}\}_{k=1}^K$ back to the central server. The server aggregates these local models into a new global model w^{t+1} by taking a weighted average based on dataset size:

$$\theta^{t+1} = \sum_{k=1}^K \frac{n_k}{N} \theta_k^{t+1} \quad (3)$$

4. Distribution and Iteration

The new global model w^{t+1} is then broadcast back to the clients to begin round $t+1$. This entire process of distribution, local updating, and global aggregation is repeated for T rounds until the model converges.

For our Federated Unlearning setting, assume that a client is requested to delete $d\%$ of class c from its knowledge base and hence reverse the learning process even from the central network.

3.1. Architecture and Initialization

Let the pre-trained global model be $f(\cdot; \Theta_G)$ with parameters Θ_G , frozen to preserve the foundational knowledge acquired during federated training. Let f_c denote the client who needs to forget $d\%$ of class c 's data.

To facilitate unlearning, we inject LoRA [15] layers into the weight matrices $W \in \mathbb{R}^{d \times k}$ corresponding to the later layers of f_c . The trainable parameters are hence, restricted to the low-rank matrices $\phi = \{A, B\}$, where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$, with the rank $r \ll \min(d, k)$.

The modified forward pass for any adapted layer can be expressed as:

$$h = Wx + \frac{\alpha}{r} (BA)x \quad (4)$$

where α is a scaling factor and x is the input vector. This structure allows the model to learn an additive ‘‘interference signal’’ (BA) while keeping the original knowledge W intact. We also introduce similar LoRA layers in the global model to introduce the effect of unlearning into the global model.

3.2. Initial Idea: Dual Loss Objective

Upon receiving a request for unlearning, the target client f_c utilizes its local forget set D_f , containing $d\%$ of the local samples of class c available to f_c and a local retain set D_r to optimize ϕ . Our initial idea was to perform a straightforward dual-objective unlearning by pushing the probability outputs of the forget samples to uniform distribution using the Kullback-Leibler divergence and standard cross entropy on the retain samples.

$$\mathcal{L}_{forget} = D_{KL}(\text{Softmax}(f(x \in D_f; \Theta_G, \phi)) \parallel \mathcal{U}) \quad (5)$$

$$\mathcal{L}_{retain} = - \sum_{(x,y) \in D_r} y \log(f(x; \Theta_G, \phi)) \quad (6)$$

$$\mathcal{L}_{overall} = \mathcal{L}_{retain} + \lambda \cdot \mathcal{L}_{forget} \quad (7)$$

However, this objective was too soft and failed to even out the outputs of samples from D_f as the KL divergence generates softer gradients compared to the cross entropy loss on D_r . Hence, the loss function prioritizes performance on the retain set than forgetting. We validated this observation after performing a hyperparameter search on λ to help weight $\mathcal{L}_{forget}$ appropriately. For large λ values, the objective focuses only on forgetting and for smaller values, the result is no forgetting. This approach would be very unstable and needs precise hyperparameter tuning which is not desirable.

3.3. Staged Unlearning Strategy

For our working algorithm, we propose a three-stage unlearning algorithm: (i) Forget the entire knowledge base of class c from f_c (ii) Regain the performance of the model on the other classes that could have been affected by altering the common features and finally (iii) Use a dual-loss objective to include samples of class c that were not supposed to be forgotten back into the knowledge base. This approach is commonly found in machine learning where we find a basin in the parameter space that could potentially contain the solution to our problem and then optimize the parameters to approach the minimum.

Stage I: Unlearning class c

To forget the specific information corresponding to class c , we perform Gradient Ascent [33] to destroy the exact memory of samples contained in gradients by moving along the direction that maximizes the loss corresponding to the class c whose samples are to be forgotten. Let D_c denote the entire set of samples of class c that f_c was trained on.

$$\mathcal{L}_{forget-stage} = + \sum_{(x,y) \in D_c} y \log(f(x; \Theta_G, \phi)) \quad (8)$$

In our implementations, we perform this selective unlearning for 15 epochs, which takes approximately 2 minutes for the standard four-layer CNN. More details can be found in the experiments section.

Stage II: Performance Recovery on Other Classes

To mitigate the catastrophic effects of Stage I on other classes due to sharing of features, we train the adapters with cross entropy loss to minimize the following loss:

$$\mathcal{L}_{final} = - \sum_{(x,y) \in D_{rec}} y \log(f(x; \Theta_G, \phi)) \quad (9)$$

where $D_{rec} = \{x \in D_r : x \notin D_c\}$ is the set of samples from classes other than the forget class c . We keep this performance recovery stage light and train the model only for 5 epochs.

Stage III: Dual loss for forgetting and recovery

Now we utilize the original D_f and D_r forget and retain sets that were given to perform a simultaneous optimization to suppress the performance on samples in D_f that could potentially spike up and return to the initial state when we try to include the samples of c to be retained into our retain loss. Hence, the final model is obtained by training the model for 15 epochs on the following loss function for our third stage:

$$\begin{aligned} \mathcal{L}_{recovery-stage} = & - \sum_{(x,y) \in D_f} y \log(f(x; \Theta_G, \phi)) \\ & + \mu \cdot \sum_{(x,y) \in D_r} y \log(f(x; \Theta_G, \phi)) \end{aligned} \quad (10)$$

where μ is annealed over time as $\mu_{epoch+1} = 0.8 * \mu_{epoch}$

3.4. Server-Side Aggregation and Integration

Once local optimization is complete, the client transmits only the optimized low-rank matrices $\phi^* = \{A^*, B^*\}$ to the central server. For a set of K clients requesting unlearning, the server aggregates these updates:

$$\Delta W_{avg} = \frac{1}{K} \sum_{i=1}^K (B_i^* A_i^*) \quad (11)$$

The server then integrates this aggregate update into the global weights:

$$W_{unlearned} = W_{global} + \frac{\alpha}{r} \Delta W_{avg} \quad (12)$$

4. Results

In this section, we provide experimental results to validate that our approach has potential and could be used in practice. Due to unavailability of code by authors and time constraints we were unable to compare with the SOTA methods. We plan to try and compare the methods hopefully for our presentation.

4.1. Evaluation Metrics

We evaluate the quality of the unlearning process through the following primary metrics:

- **Model Utility:** Assessed via classification accuracy on a held-out global test set D_{test} .
- **Attack Success Rate** To formally evaluate the privacy leakage of the unlearned model, we employ an Attack Success Rate (ASR) metric based on class-specific confidence thresholds. For a target model M with parameters θ , we define the membership score $S(x, y; \theta)$ for a sample (x, y) as the predicted probability of the ground-truth class y . To account for variance in model confidence across different labels, we derive optimal thresholds τ_c for each class $c \in \{1, \dots, K\}$ using a shadow model trained on a disjoint but distributionally similar proxy dataset. The membership score is given by:

$$S(x, y; \theta) = \text{softmax}(M(x; \theta))_y \quad (13)$$

The ASR for a dataset D represents the fraction of samples that the attacker correctly identifies as part of the

training set. It is computed as the average of an indicator function $\mathbb{I}(\cdot)$ that outputs 1 if the confidence score exceeds the threshold for its respective class:

$$ASR(D) = \frac{1}{|D|} \sum_{(x,y) \in D} \mathbb{I}(S(x,y;\theta) \geq \tau_y) \quad (14)$$

In our unlearning evaluation, we calculate $ASR(D_{retain})$ to verify attack efficacy, $ASR(D_{test})$ as a baseline for unseen data, and $ASR(D_{forget})$ to measure unlearning success. An ideal unlearning mechanism results in $ASR(D_{forget}) \approx ASR(D_{test})$, indicating that the forgotten samples are indistinguishable from samples never encountered during training.

4.2. Comparisons with the Retrain Strategy

We use a four-layer CNN for all our models (clients and global). Our experiments were conducted on the CIFAR-10 dataset [18]. We chose to forget 30% of samples from one class of one client as we feel this is representative of the typical results an inference provider could face. Some entity (company or person) could ask for the removal of their data from the knowledge base of a model and this should generally apply to one or two classes. Hence, we choose $d = 30\%$ for our baseline experiments.

As discussed earlier, the gold standard for any unlearning algorithm would be the naive retraining strategy which initializes a model and trains it from scratch on only D_r after excluding D_f . We trained a model following the same strategy. We present the results in table 1. As demonstrated by the accuracy values, our method shows strong retention of knowledge from the retain set while minimizing the performance on the forget set.

Table 1. Comparison of Model Unlearning Techniques

Model	Model Accuracy Before Unlearn	Model Acc After Unlearn	Acc on Retain Set	Acc on Forget Set	ASR
Naive Retraining	81.53%	79.43	84.61	87.421	0.8245
KL Based Unlearning	81.53%	74.36	74.79	86.16	0.166
3 Staged	81.53%	77.13	78.35	27.04	0.1925

Surprisingly, even the naive retraining algorithm is able to generalize enough to perform well on the forget set, which it has never seen before. This raises the question if accuracy on the forget set can be considered a valid metric as clearly the samples from D_f were not used to train our Naive Retrain model and yet we get good performance on the same. We then compute the ASR metric to check for membership inference and again we are met with a high ASR, for which lower is supposed to be better. Hence, **we find a need for reliable metrics to conclude if a certain sample was used for training a model or not.** We find our

final model is able to maintain high test set accuracy while keeping accuracy on the forget set very low compared to Naive Retraining and KL based Unlearning.

Table 2. Effect of λ on Unlearning Performance

λ	testacc	forget acc	retain acc	ASR
0.2	72.32	91.80	72.32	0.28
0.5	74.02	90.56	74.05	0.284
0.7	74.36	86.16	74.79	0.166
1	74.01	80.50	75.54	0.171
5	72.96	72.32	74.30	0.167

We present the ablation study performed on the λ values of our first KL divergence based method in table 2. As seen, the overall training regimen is very unstable and gives results that vary heavily as we increase the weight for $\mathcal{L}_{forget}$.

Table 3. Comparison of LoRA Adaptation Strategies

Model	Test Acc	Forget Set acc	Retain Set acc	ASR
All layers lora	74.46	13.84	77.11	0.250
Last stage + classifiers	77.13	27.04	78.35	0.192

Finally, table 3 shows that applying LoRA to the last layers of the network performs better, which is in line with our intuition that disturbing earlier layers can disrupt the performance of the model on all classes as the earlier layers of a neural network extracts low level information that is common to all classes.

5. Conclusion

In this work, we presented a novel federated unlearning framework that leverages Low-Rank Adaptation (LoRA) to enable efficient, reversible erasure of targeted client data from a global model trained via FedAvg on CIFAR-10. Our three-stage strategy—gradient ascent for class-specific forgetting, lightweight recovery on unaffected classes, and dual-loss refinement—achieves post-unlearning test accuracy of 77.13% (from 81.53% baseline), 78.35% retain set accuracy, sharply reduced forget set accuracy of 27.04%, and low ASR of 0.1925, outperforming naive KL-divergence unlearning while sidestepping computationally prohibitive retraining.

5.1. Efficiency and Future Work

Future work will focus on expanding evaluations to more complex architectures and comparing this approach against current state-of-the-art federated unlearning methods. An interesting direction of research is to develop accurate metrics that can determine the inclusion of samples in the training data of a model reliably.

References

- [1] Thomas Baumhauer, Pascal Schöttle, and Matthias Zepelzauer. Machine unlearning: Linear filtration for logit-based classifiers. *Machine Learning*, 111(9):3203–3226, 2022. 2
- [2] Lucas Bourtole, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. *CoRR*, abs/1912.03817, 2019. 1, 2
- [3] Jonathan Brophy and Daniel Lowd. Machine unlearning for random forests. In *International conference on machine learning*, pages 1092–1104. PMLR, 2021. 2
- [4] Yinzhi Cao and Junfeng Yang. Towards making systems forget with machine unlearning. In *2015 IEEE symposium on security and privacy*, pages 463–480. IEEE, 2015. 2
- [5] Tianshi Che, Yang Zhou, Zijie Zhang, Lingjuan Lyu, Ji Liu, Da Yan, Dejing Dou, and Jun Huan. Fast federated machine unlearning with nonlinear functional theory. In *International conference on machine learning*, pages 4241–4268. PMLR, 2023. 2
- [6] Xiangshan Gao, Xingjun Ma, Jingyi Wang, Youcheng Sun, Bo Li, Shouling Ji, Peng Cheng, and Jiming Chen. Verifi: Towards verifiable federated unlearning. *IEEE Transactions on Dependable and Secure Computing*, 21(6):5720–5736, 2024. 2
- [7] Hanlin Gu, Gongxi Zhu, Jie Zhang, Xinyuan Zhao, Yuxing Han, Lixin Fan, and Qiang Yang. Unlearning during learning: An efficient federated machine unlearning method. *arXiv preprint arXiv:2405.15474*, 2024. 2
- [8] Qi Guo, Di Wu, Yong Qi, Saiyu Qi, and Qian Li. Flmjr: Improving robustness of federated learning via model stability. In *European Symposium on Research in Computer Security*, pages 405–424. Springer, 2022. 2
- [9] Qi Guo, Yong Qi, Saiyu Qi, Di Wu, and Qian Li. Fedmcsa: Personalized federated learning via model components self-attention. *Neurocomputing*, 560:126831, 2023. 2
- [10] Qi Guo, Di Wu, Yong Qi, and Saiyu Qi. Dual class-aware contrastive federated semi-supervised learning. *IEEE Transactions on Mobile Computing*, 24(2):1073–1089, 2024. 2
- [11] Varun Gupta, Christopher Jung, Seth Neel, Aaron Roth, Saeed Sharifi-Malvajerdi, and Chris Waites. Adaptive machine unlearning. *Advances in Neural Information Processing Systems*, 34:16319–16330, 2021. 2
- [12] Seok-Ju Hahn and Junhye Lee. Privacy-preserving federated bayesian learning of a generative model for imbalanced classification of clinical data. *arXiv preprint arXiv:1910.08489*, 2019. 2
- [13] Anisa Halimi, Swanand Kadhe, Ambrish Rawat, and Nathalie Baracaldo. Federated unlearning: How to efficiently erase a client in fl?, 2023. 1, 2
- [14] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey, 2024. 2
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. 1, 3
- [16] Hyejun Jeong, Shiqing Ma, and Amir Houmansadr. A survey on federated unlearning: Challenges and opportunities, 2025. 1
- [17] Peter Kairouz and H Brendan McMahan. Advances and open problems in federated learning. *Foundations and trends in machine learning*, 14(1-2):1–210, 2021. 2
- [18] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. *Master's thesis, Department of Computer Science, University of Toronto*, 2009. 1, 5
- [19] Chunxiao Li, Haipeng Jiang, Jiankang Chen, Yu Zhao, Shuxuan Fu, Fangming Jing, and Yu Guo. An overview of machine unlearning. *High-Confidence Computing*, 5(2):100254, 2025. 1
- [20] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2:429–450, 2020. 2
- [21] Zelei Liu, Yuanyuan Chen, Yansong Zhao, Han Yu, Yang Liu, Renyi Bao, Jinpeng Jiang, Zaiqing Nie, Qian Xu, and Qiang Yang. Contribution-aware federated learning for smart healthcare. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 12396–12404, 2022. 2
- [22] Ziyao Liu, Yu Jiang, Jiyuan Shen, Minyi Peng, Kwok-Yan Lam, Xingliang Yuan, and Xiaoning Liu. A survey on federated unlearning: Challenges, methods, and future directions, 2024. 1, 2
- [23] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data, 2023. 1
- [24] Dinh C Nguyen, Quoc-Viet Pham, Pubudu N Pathirana, Ming Ding, Aruna Seneviratne, Zihuai Lin, Octavia Dobre, and Won-Joo Hwang. Federated learning for smart healthcare: A survey. *ACM Computing Surveys (Csur)*, 55(3):1–37, 2022. 2
- [25] Zibin Pan, Zhichao Wang, Chi Li, Kaiyan Zheng, Boqi Wang, Xiaoying Tang, and Junhua Zhao. Federated unlearning with gradient descent and conflict mitigation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 19804–19812, 2025. 2
- [26] R Shao et al. Stochastic channel-based federated learning for medical data privacy preserving (2019). *arXiv preprint arXiv:1910.11160*. 2
- [27] Junxiao Wang, Song Guo, Xin Xie, and Heng Qi. Federated unlearning via class-discriminative pruning. In *Proceedings of the ACM web conference 2022*, pages 622–632, 2022. 2
- [28] Leijie Wu, Song Guo, Junxiao Wang, Zicong Hong, Jie Zhang, and Yaohong Ding. Federated Unlearning: Guarantee the Right of Clients to Forget. *IEEE Network*, 36(5):129–135, 2022. 1
- [29] Wei Yuan, Hongzhi Yin, Fangzhao Wu, Shijie Zhang, Tiek He, and Hao Wang. Federated unlearning for on-device recommendation. In *Proceedings of the sixteenth ACM international conference on web search and data mining*, pages 393–401, 2023. 2

- [30] Lefeng Zhang, Tianqing Zhu, Haibin Zhang, Ping Xiong, and Wanlei Zhou. Fedrecovery: Differentially private machine unlearning for federated learning frameworks. *IEEE Transactions on Information Forensics and Security*, 18: 4732–4746, 2023. [2](#)
- [31] Yian Zhao, Pengfei Wang, Heng Qi, Jianguo Huang, Zongzheng Wei, and Qiang Zhang. Federated unlearning with momentum degradation. *IEEE Internet of Things Journal*, 11(5):8860–8870, 2023. [2](#)
- [32] Yuhao Zhou, Qing Ye, and Jiancheng Lv. Communication-efficient federated learning with compensated overlap-fedavg. *IEEE Transactions on Parallel and Distributed Systems*, 33(1):192–205, 2022. [2](#)
- [33] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2019. [4](#)